LAB 5

STATIC CODE ANALYSIS

Car Rental System | Python Quality, Security and Style

Student	Pranav Shashikiran Atreya
Student ID / SRN	PES2UG24CS912
Class / Section	5D
Course / Lab	SWE Lab
Repository	https://github.com/pranav-atreya/L5_56_car-rental

Outcome summary

PYLINT	BANDIT	FLAKE8
10.00 / 10	**No issues identified**	**0 violations**

This report documents the original program behaviour, baseline static-analysis findings, corrective changes, final verification results, and reflections on the use of static-analysis tools.

1. Objective and program overview

The objective of this lab was to use Pylint, Bandit, and Flake8 to examine a Python program without relying only on running it, document genuine issues, correct the code, rerun the tools, and reflect on the value and limitations of static analysis.

Program overview

The supplied car-rental demonstration maintains a vehicle fleet, registers customers in an in-memory SQLite database, creates and returns bookings, calculates extra-kilometre charges, applies promotional pricing rules, validates registration plates, hashes licence identifiers, checks an administrator login, lists available vehicles, and writes a receipt file.

2. Environment and procedure

Item	Environment / method
Operating environment	Windows PowerShell in VS Code
Python	3.13.1 (visible in Bandit output)
Pylint version	Not visible in the screenshots supplied
Bandit version	1.9.4 documentation path visible in the baseline report
Flake8 version	Not visible in the screenshots supplied
Source repository	https://github.com/SETAPESU26/L5_56_car-rental

Procedure followed

1. Ran the original demonstration and observed its printed output before changing the source.
2. Created and activated a Python virtual environment and installed Pylint, Bandit, and Flake8.
3. Ran the three analysis tools against car_rental_original.py and saved pylint_before.txt, bandit_before.txt, and flake8_before.txt.
4. Compared car_rental_original.py with the corrected car_rental.py and reviewed security, reliability, validation, and style changes.
5. Ran the corrected program, then reran all three tools against car_rental.py and saved the final reports.
6. Corrected the remaining Flake8 line-length and trailing-whitespace findings; the final captured Flake8 report was empty.

3. Baseline and final analysis results

The table below records only results supported by the screenshots supplied. The baseline Pylint screenshot shows its finding list, but not the score summary at the end of that report.

Tool	Baseline findings visible in evidence	Final result shown in evidence
Pylint	Multiple findings including C0301 (long lines), C0114/C0116 (missing docstrings), C0103 (function naming), W0102 (mutable default), W0702 (bare except), W0122 (exec), W1514 (open without encoding), and R1732 (use with). Original score was not visible.	10.00/10 in pylint_after.txt.
Bandit	The captured portion shows B608 possible SQL injection at customer registration and lookup (Medium severity), and B101 use of assert (Low severity). The screenshot is a visible excerpt of the baseline report.	"No issues identified"; 180 lines scanned, 0 issues by severity/confidence in bandit_after.txt.
Flake8	Captured findings include F401 unused json import; E302 blank-line rules; E501 overlong lines; E722 bare except; E712 comparison to True; and E305 blank lines after function/class definitions.	Final flake8_after.txt is empty: no reported violations.

Interpretation

- Pylint checks general code quality and design/style practices. The corrected source received the screenshot-supported score of 10.00/10.
- Bandit focuses on security patterns. The original screenshot visibly reports SQL injection risks; the corrected scan reports no issues for the 180 lines scanned.
- Flake8 initially reported formatting/style violations. The intermediate outputs show two E501 line-length findings, then one W291 trailing-whitespace finding; those were corrected before the final empty report.
- Static-analysis results complement, rather than replace, the runtime demonstration and manual review. Some functional defects required reasoning about the program's output and booking flow.

4. Issue register and fixes

The original locations below refer to the supplied car_rental.py before the fixes. Rule IDs are included only where supported by the captured tool output; not every source-level defect is guaranteed to be flagged by each tool.

Issue	Type	Original location	Evidence / effect	Fix applied
SQL injection in customer registration and lookup	Security	23; 26–27	Bandit B608 (Medium) visibly reported twice. User-supplied text was interpolated or concatenated into SQL.	Use SQLite placeholders (?) and pass input as query parameters.
Dynamic execution in promotion function	Security	50–54	Pylint W0122 flags the use of exec. The function executed a string as Python code.	Replace exec with a fixed allowlist of named promotion calculations; reject unsupported rules.
Unsafe receipt-file creation and permissions	Security	62–68	Source review: tempfile.mktemp() has a race risk and chmod 0777 grants overly broad permissions on POSIX systems.	Use tempfile.mkstemp(), write through os.fdopen with UTF-8, clean up if writing fails, and do not grant world-open permissions.
Mutable default list	Reliability / quality	16–19	Pylint W0102 flags features=[]. The same list could be shared across calls.	Default to None and create a fresh list inside the function.
Overly broad exception handling	Reliability / style	36–43	Pylint W0702 and Flake8 E722 identify bare except handling. Errors were silently swallowed.	Validate inputs and raise a meaningful, specific ValueError for expected invalid cases.
Invalid rental dates	Functional bug	29–34; demo line 90	The original demo printed “Rent Innova: -9000” because end date preceded start date.	Parse both dates and reject a duration of zero or less before pricing.
Duplicate booking / availability state	Functional bug	29–34; demo line 89	The original demo rented the Swift again because its availability was not checked/updated.	Reject bookings for unavailable cars and set availability to False on a successful booking.
Negative daily rate	Input validation	Demo line 85	The original demo added a Nexon with a daily rate of -500.	Reject negative daily rates when adding a car.
Weak/predictable security primitives and embedded credential	Security design	44–48; 70–73	The original used random.randint for booking codes, SHA-1 for licence identifiers, and a hard-coded default admin password.	Use secrets for booking codes, SHA-256 for the demonstration hash, and environment-configured admin credentials with constant-time comparison.
Style and maintainability issues	Style / maintainability	Multiple; including import line 2 and main line 101	Pylint and Flake8 report unused import, naming, docstring, whitespace/line-length and main-guard-related quality problems.	Use snake_case, docstrings and standard formatting; remove unused imports and guard the entry point with if __name__ == "__main__".

5. Code changes: SQL injection and arbitrary execution

Change 1 — Parameterised SQL

Original pattern: SQL statements were assembled using customer-controlled strings. Bandit B608 identified possible SQL injection in both customer registration and lookup (Medium severity).

```
# Before: registration
conn.execute("INSERT INTO customers VALUES ('%s', '%s', '%s')" %
          (name, licence, phone))

# Before: lookup
query = "SELECT * FROM customers WHERE name = '" + name + "'"
return conn.execute(query).fetchall()
```

```
# After: registration
conn.execute(
   "INSERT INTO customers (name, licence, phone) VALUES (?, ?, ?)",
   (name, licence, phone),
)

# After: lookup
return conn.execute(
   "SELECT * FROM customers WHERE name = ?", (name,)
).fetchall()
```

Why this is safer: SQLite receives the SQL structure and the user's values separately. A value such as x' OR '1'='1 is treated as a literal name, not as SQL syntax. In the final demo, the injection-style lookup returns an empty list: [].

Change 2 — Remove exec from promotion handling

The original applyPromo(rule, amount) used exec(rule, {}, scope), which treated the supplied rule as executable Python. Pylint reported W0122 (exec-used).

```
# Before
scope = {"amount": amount}
exec(rule, {}, scope)
return scope["amount"]
```

```
# After (simplified)
supported_rules = {
   "10_percent_off": lambda value: value * 0.9,
   "20_percent_off": lambda value: value * 0.8,
}
# Select only a supported name; raise ValueError otherwise.
calculation = supported_rules[rule]
return calculation(amount)
```

Why this is safer: the revised function chooses a calculation from a finite allowlist and rejects unsupported promotion names. It no longer executes arbitrary text. The demo uses the supported name 10_percent_off and calculates 4,050.0 from 4,500.

6. Code changes: receipts, defaults and booking validation

Change 3 — Create receipts safely

The original receipt writer used tempfile.mktemp(), opened the chosen path without a context manager or explicit encoding, and then set permissions to 0o777. The corrected function uses an already-created temporary file descriptor and removes the file if writing fails.

```
# Before
path = tempfile.mktemp(suffix=".txt")
f = open(path, "w")
f.write(receipt_text)
f.close()
os.chmod(path, 0o777)
return path
```

```
# After (core pattern)
file_descriptor, path = tempfile.mkstemp(
    suffix=".txt", prefix="car_rental_receipt_"
)
try:
    with os.fdopen(file_descriptor, "w", encoding="utf-8") as receipt_file:
        receipt_file.write(receipt_text)
except Exception:
    Path(path).unlink(missing_ok=True)
    raise
return path
```

Why this is safer: the file is created atomically by the tempfile API, the file is managed with a context manager and explicit UTF-8 encoding, and the code avoids world-open permissions.

Change 4 — Remove a mutable default argument

```
# Before
def addCar(plate, model="Unknown", daily_rate=0, features=[]):
    ...

# After
def add_car(plate, model="Unknown", daily_rate=0, features=None):
    if features is None:
        features = []
```

Why this is better: each call that omits features receives its own new list rather than sharing a single mutable default. The function was also renamed to snake_case and given a docstring.

Additional functional fixes

- The program now rejects negative daily rates and invalid or non-positive rental periods.
- Booking requires a registered customer, a known car, and current availability. Successful bookings mark the car unavailable to prevent overlap.
- Returning an unknown vehicle or a negative kilometre value raises a meaningful error instead of being silently ignored.
- The licence demonstration hash uses SHA-256; booking codes use secrets; the admin credential is read from environment variables, not a source-code default.

7. Runtime verification and conclusion

Observed final runtime checks

Check	Observed final behaviour
Invalid car rate	Nexon with daily rate -500 rejected with “Daily rate cannot be negative.”
Valid rental	Swift rental displayed a charge of 4500.
Overlapping rental	Second Swift booking rejected because the car was not available.
Invalid date range	Rental with end date before start date rejected.
SQL injection-style lookup	Lookup returned [] rather than all customer records.
Unknown vehicle return	Rejected with an unknown registration plate error.
Promotion	Supported 10 percent promotion returned 4050.0 for 4500.
Program completion	The demonstration completed and printed the receipt path.

Final outcome

The corrected program’s final screenshots show Pylint at 10.00/10, Bandit reporting “No issues identified” with 180 lines scanned, and an empty final Flake8 report. These results support successful static-analysis cleanup for the submitted demonstration, but they do not prove that the program is free of every possible defect. Runtime tests, code review, and a more complete production security design would still be necessary.

8. Reflection questions

1. What is static code analysis, and why is it useful?

Static code analysis inspects source code without running the program. It helps detect suspicious constructs, likely bugs, security risks, and style or maintainability problems early, before software is deployed. In this lab, Pylint highlighted maintainability issues and risky code patterns, Bandit identified SQL-injection risks in the original program, and Flake8 reported formatting and linting violations. The tools complement one another, but none guarantees that the program is fully correct or secure.

2. What issues did you identify and fix?

The original program interpolated customer inputs into SQL, executed a supplied promotion-rule string using exec, created receipts through tempfile.mktemp and overly broad permissions, and used a mutable list default. It also silently swallowed exceptions, accepted a negative daily rate and an end date before the start date, and could book a car more than once because availability was not updated. The revised program uses parameterised SQL, a fixed set of promotions, safe temporary-file handling, a None default for the feature list, explicit validation, clearer exceptions, and updated availability. It also improves token generation, hashing, credential handling, naming, formatting, and module execution structure. The final evidence showed Pylint 10.00/10, no Bandit issues identified, and an empty Flake8 report.

3. What was challenging, and how did you address it?

The main challenge was distinguishing style findings from problems that affect security or real program behaviour. For example, the original demonstration showed a negative rental cost when the end date came before the start date; this is a functional bug, not merely a style issue. Another challenge was to verify that corrections worked instead of assuming that a changed line was enough. The approach was to compare the original and revised source, run the demonstration before and after, use the three tools, and iterate on Flake8 output until the final report was empty. This also reinforced that a tool's silence is not a replacement for tests and human review.

4. How does static analysis improve software quality and maintainability?

Static analysis makes code review repeatable and gives developers concrete locations and rule identifiers to investigate. It helps teams enforce consistent naming and formatting and identify risky patterns before release. It is most useful when paired with code review and runtime tests: some findings may be false positives, and tools cannot reason perfectly about every business rule or execution path. In this lab, the tools found code-level risks, while running the demonstration revealed the consequences of invalid dates and duplicate bookings.

Conclusion

The lab improved the demonstration car-rental system by addressing injection risks, unsafe dynamic execution, temporary-file handling, mutable state, input validation, booking availability, and style. The final captured results were Pylint 10.00/10, Bandit with no issues identified, and Flake8 with no reported violations. The exercise showed why multiple static-analysis tools, manual inspection, and runtime behaviour checks should be used together.

Appendix A — Original execution

The original output exposes behaviour that should be rejected: the program rents the Swift again and prints a negative Innova rental cost, and the injection-style lookup returns both customer records.

```
(.venv) PS C:\Users\Pranav S Atreya\PES\sem5\SWE\lab\Lab5_Car_Rental_Deliverables> python .\car_rental_original.py
Rent Swift: 4500
Rent Swift again: 1500
Rent Innova: -9000
Lookup injection: [('Ravi', 'DL-0420110012345', '9845012345'), ('Meera', 'DL-0420110099999', '9845099999')]
Extra km charge: 600
Return unknown car: None
Promo price: 4050.0
Plate valid? True
Licence hash: acb5ec2096ab233046fd883ca20b72a72a1a9354
Admin: True
Available: ['KA01AB1234', 'KA02CD5678', 'KA03EF9999']
Receipt saved to: C:\Users\PRANAV~1\AppData\Local\Temp\tmpb4kz1s76.txt
(.venv) PS C:\Users\Pranav S Atreya\PES\sem5\SWE\lab\Lab5_Car_Rental_Deliverables>
```

Figure 1. Original car-rental demonstration before corrections.

Appendix B — Pylint baseline output

Evidence captured from the Windows PowerShell / VS Code workflow.

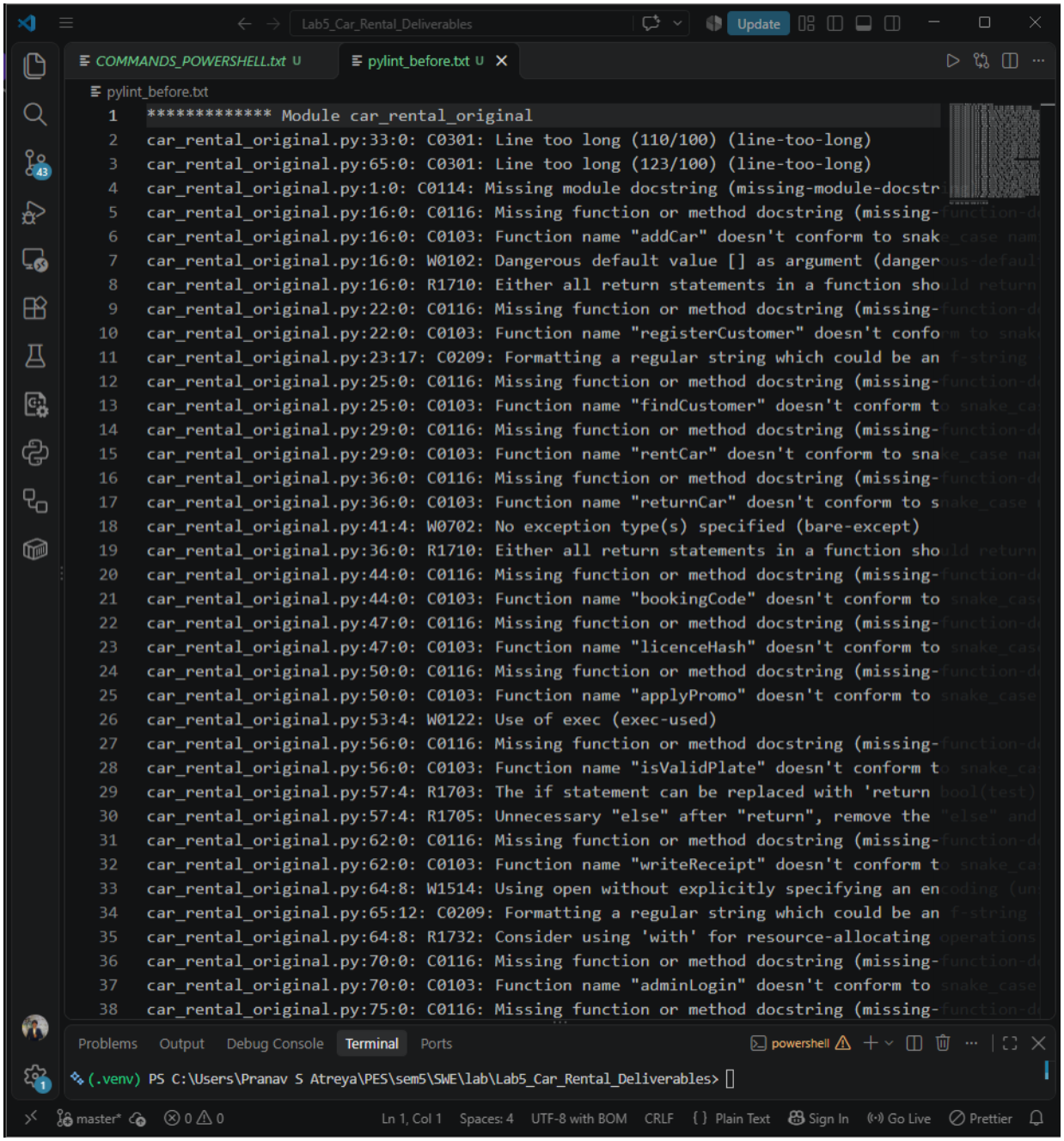


Figure 2. Original Pylint findings, including the mutable default, bare except, exec use, naming, docstring, and line-length warnings.

Appendix C — Bandit baseline output

Evidence captured from the Windows PowerShell / VS Code workflow.

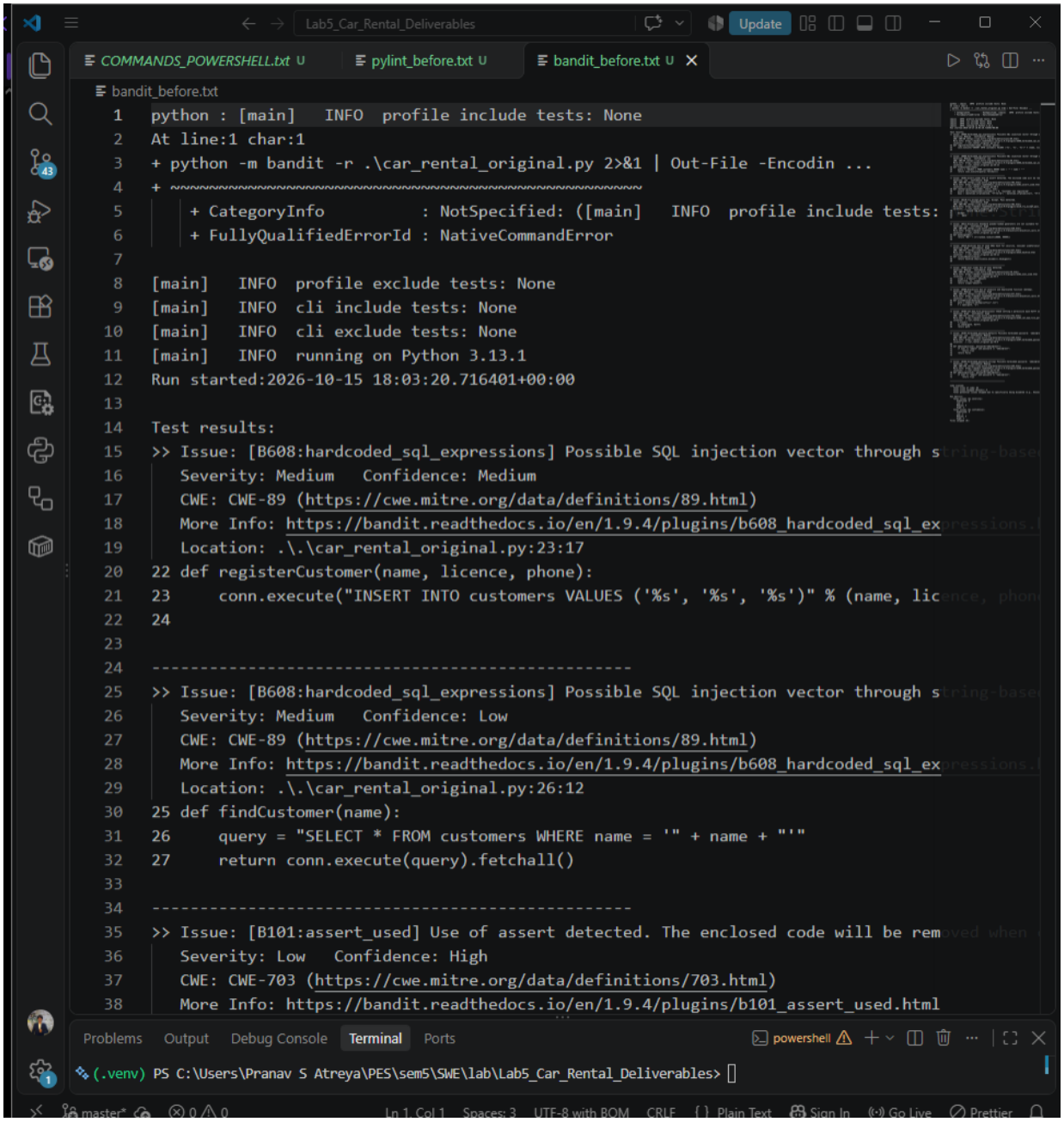


Figure 3. Original Bandit output visibly shows two B608 possible SQL-injection findings at Medium severity and a B101 assert finding at Low severity.

Appendix D — Flake8 baseline output

Evidence captured from the Windows PowerShell / VS Code workflow.

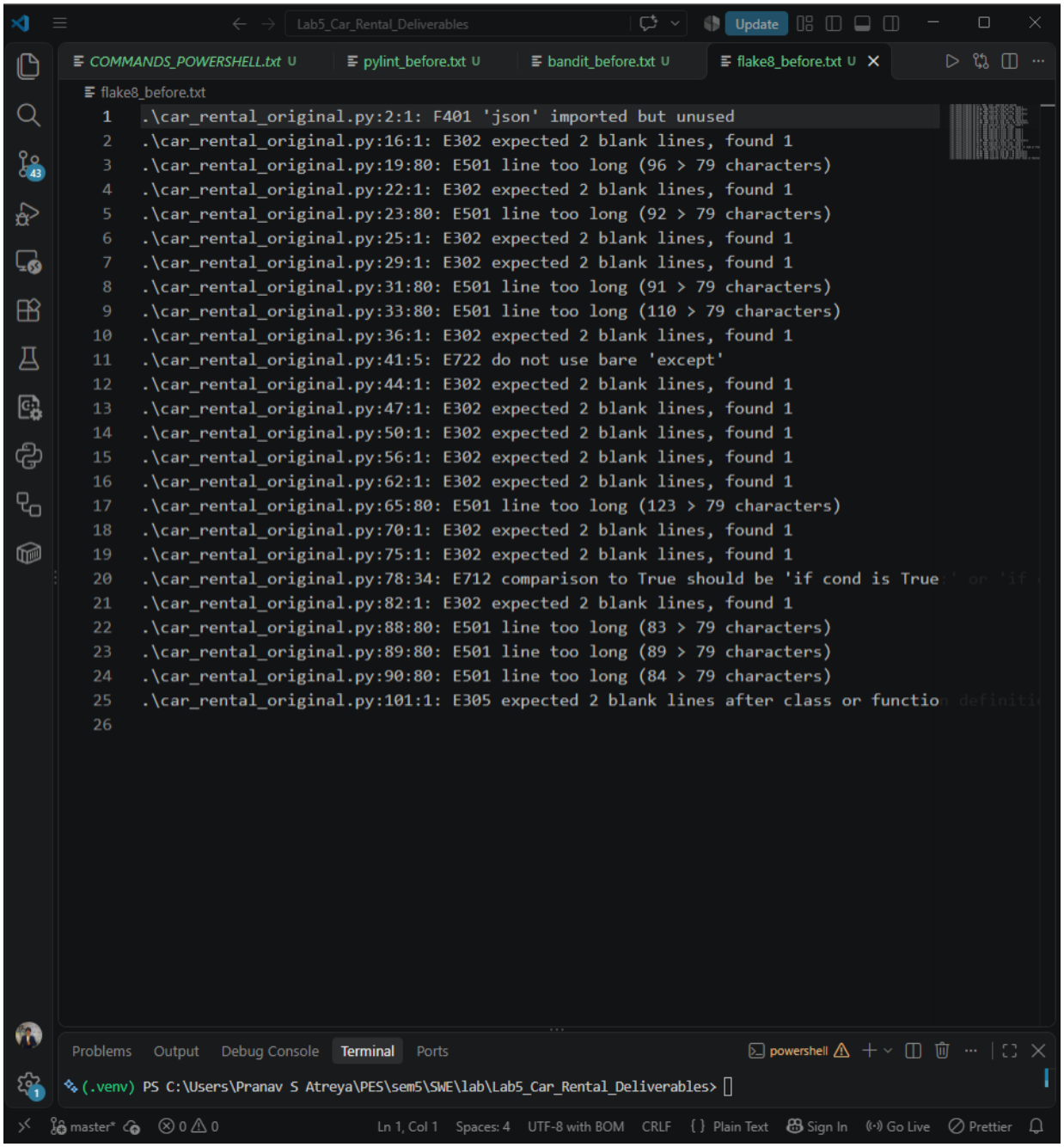


Figure 4. Original Flake8 findings include F401, E302, E501, E722, E712, and E305.

Appendix E — Source comparison (1 of 2)

Evidence captured from the Windows PowerShell / VS Code workflow.

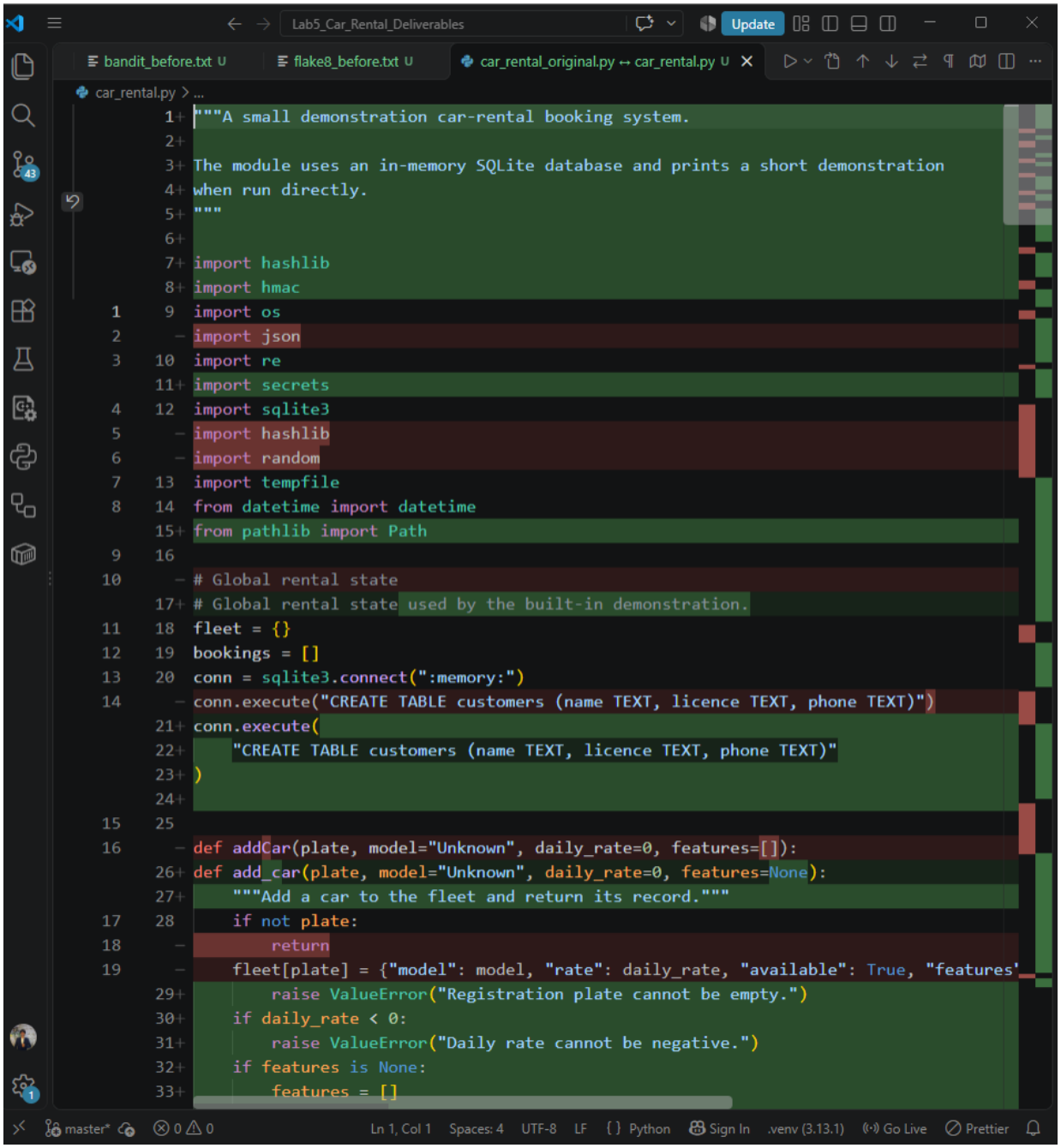


Figure 5. VS Code comparison showing the module docstring, import cleanup, mutable-default correction, and initial validation in add_car().

Appendix F — Source comparison (2 of 2)

Evidence captured from the Windows PowerShell / VS Code workflow.

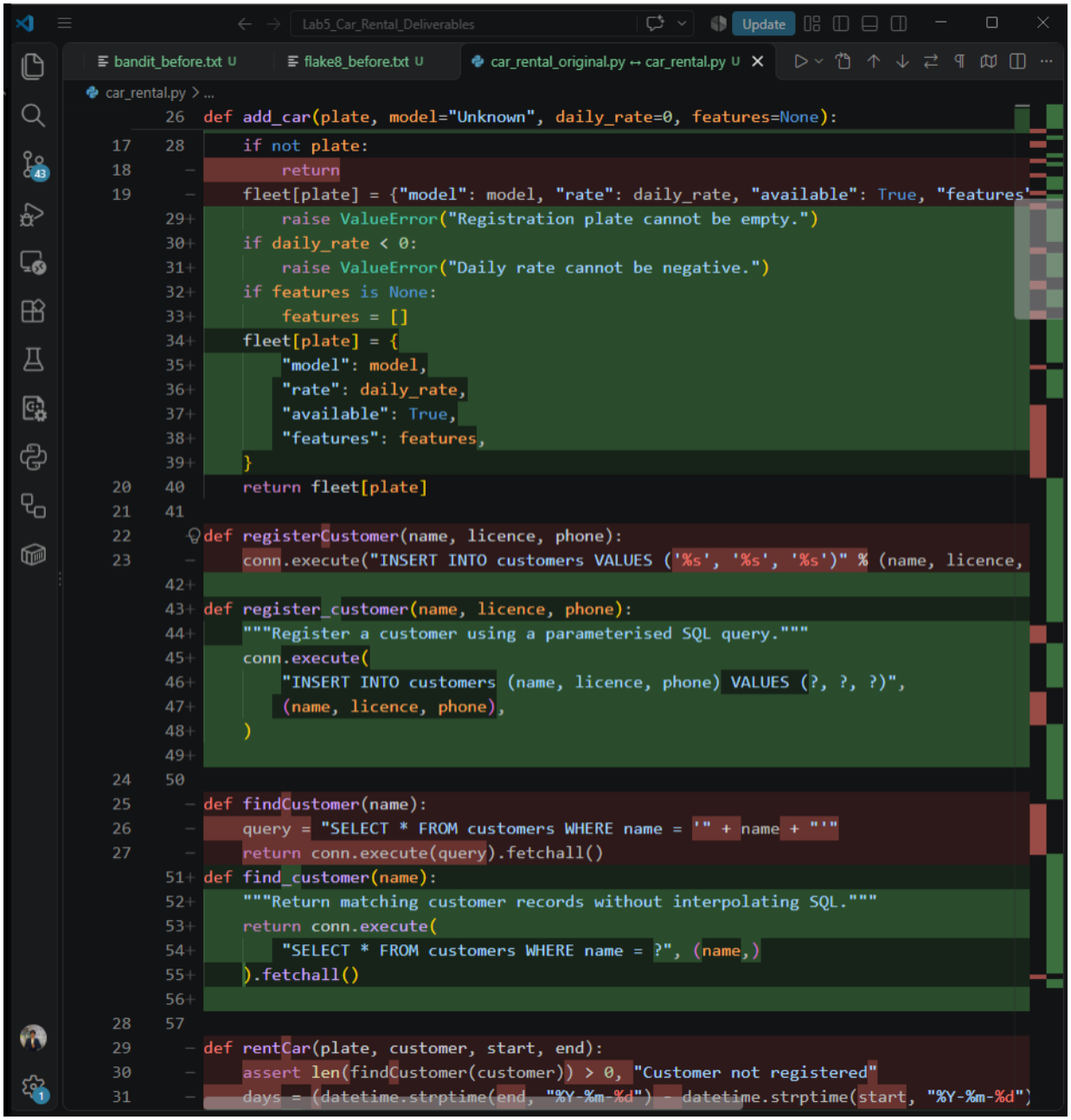


Figure 6. VS Code comparison showing parameterised SQL for customer registration and lookup.

Appendix G — Corrected runtime output

The revised run rejected the negative rate, an overlapping booking, invalid rental dates, and an unknown vehicle return. The SQL-injection-style lookup returned an empty list.



Figure 7. Final car-rental demonstration after the code changes.

Appendix H — Final Pylint output

Evidence captured from the Windows PowerShell / VS Code workflow.

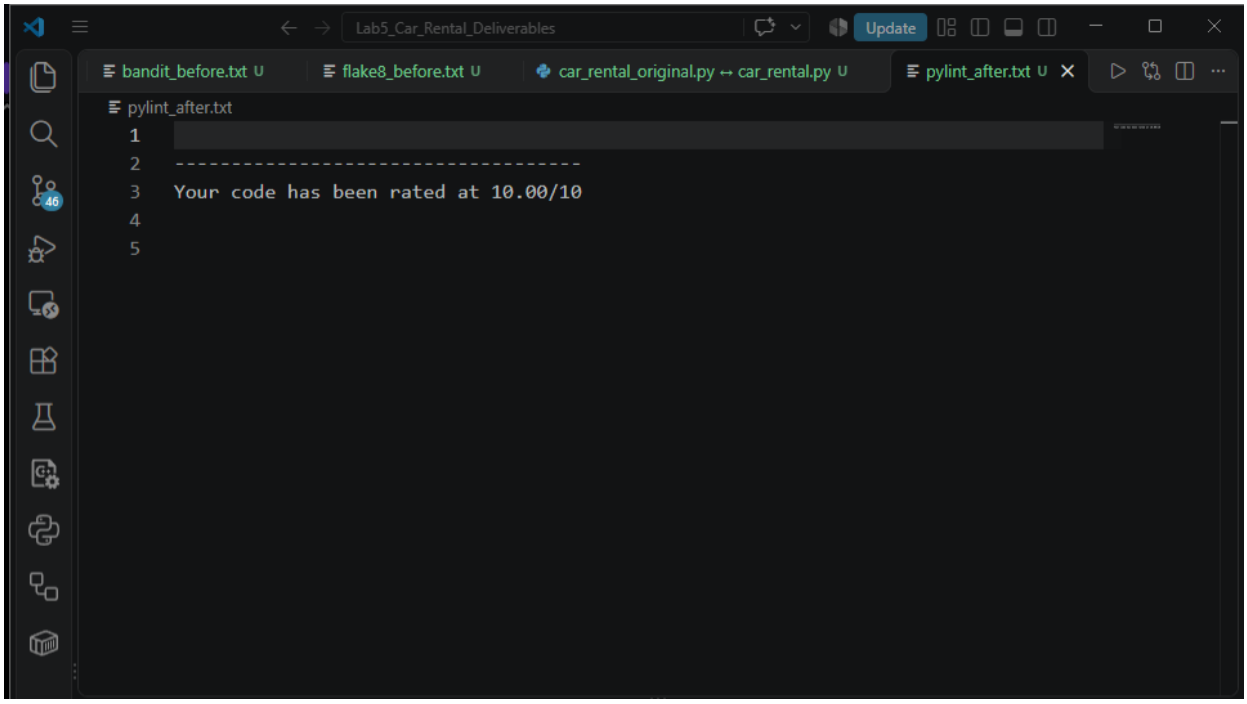


Figure 8. Final Pylint output reports a score of 10.00/10.

Appendix I — Final Bandit output

Evidence captured from the Windows PowerShell / VS Code workflow.

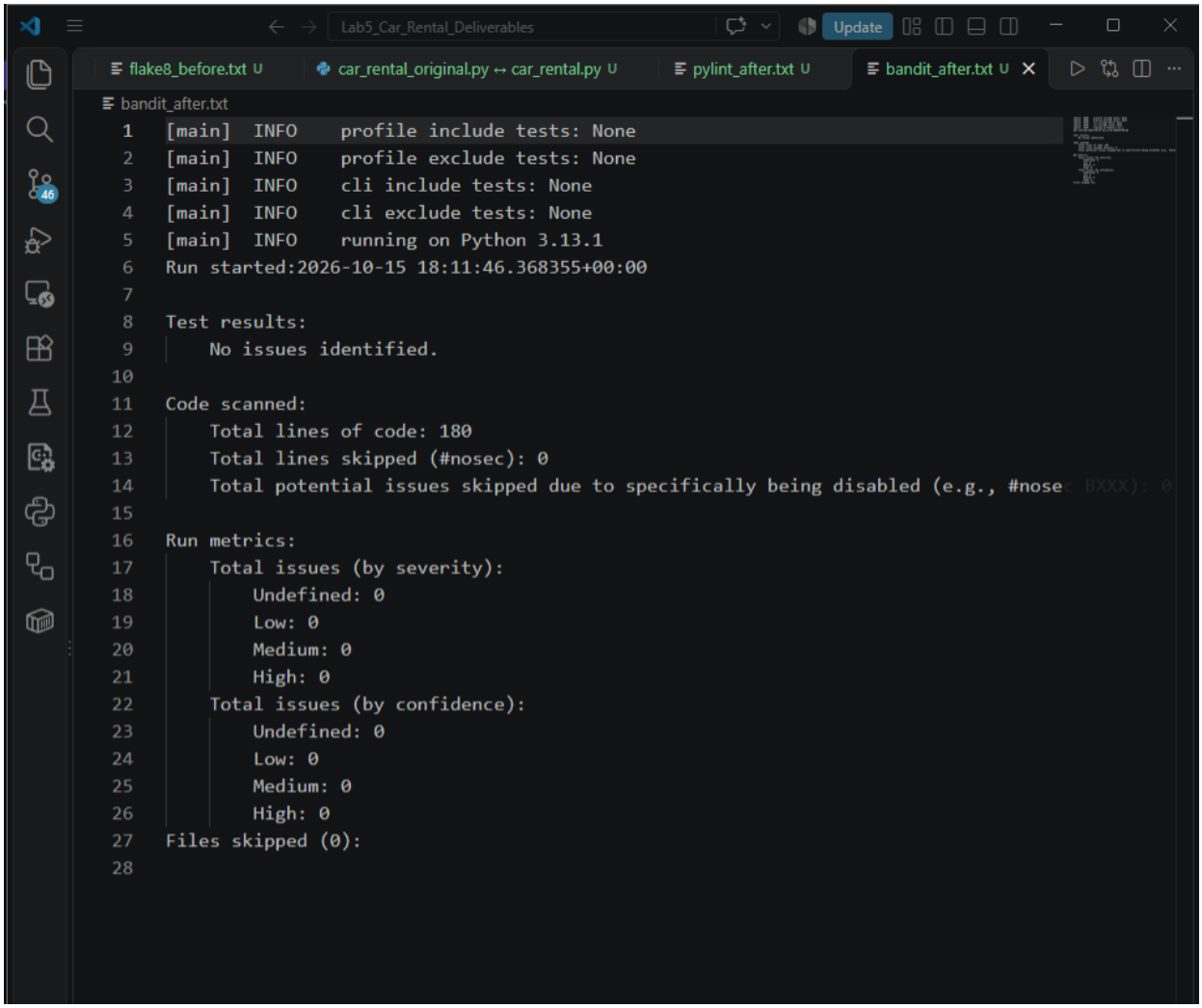


Figure 9. Final Bandit output: no issues identified; 180 lines of code scanned; zero reported issues by severity and confidence.

Appendix J — Flake8 cleanup iterations

The screenshots below document the final cleanup loop: the first rerun showed two E501 line-length violations, and after those were corrected a single W291 trailing-whitespace violation remained. The trailing whitespace was removed before the final scan.

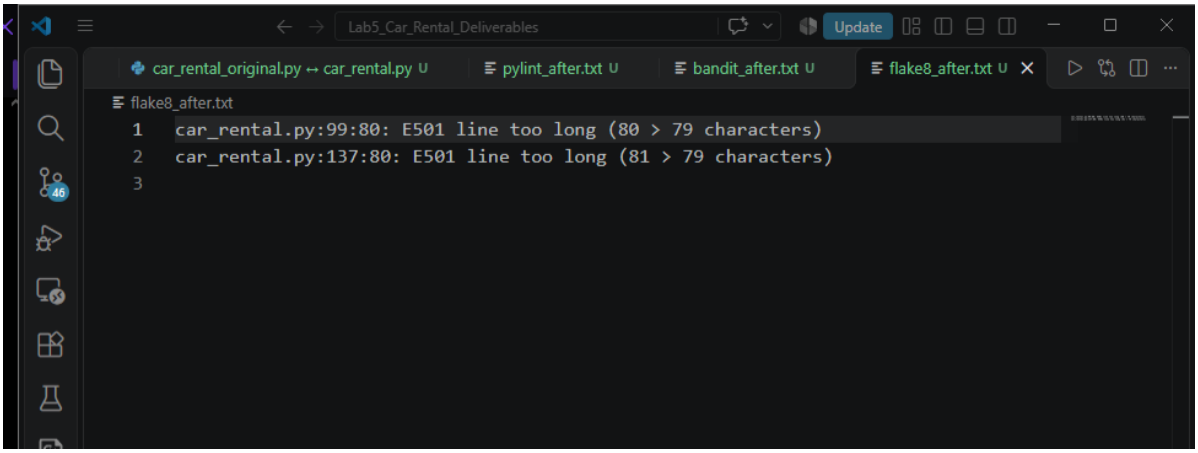


Figure 10. Intermediate Flake8 output showing two E501 line-length violations.



Figure 11. Intermediate Flake8 output showing one remaining W291 trailing-whitespace violation.

Appendix K — Final Flake8 output

Evidence captured from the Windows PowerShell / VS Code workflow.

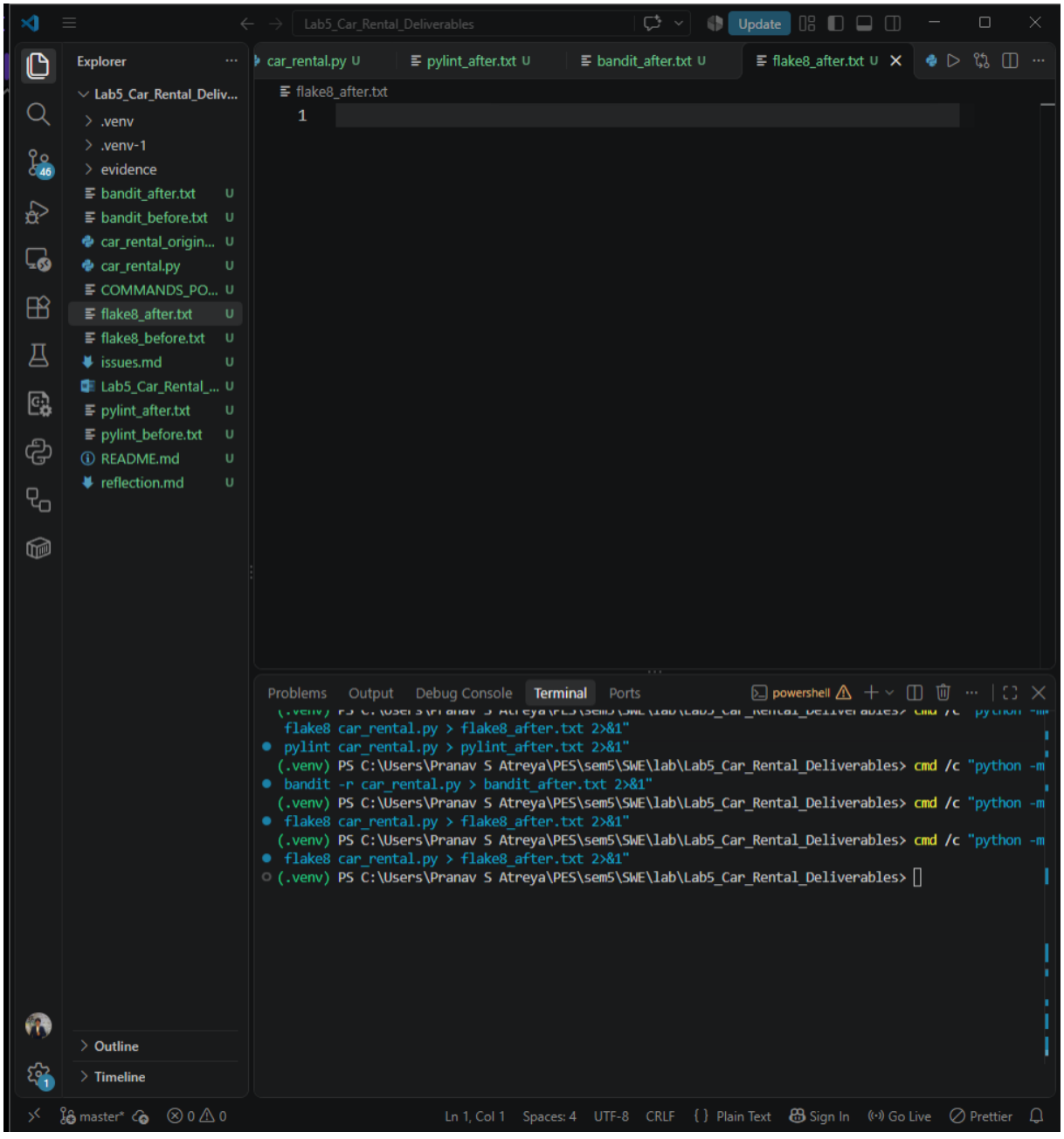


Figure 12. Final Flake8 output is empty after the remaining whitespace was removed.